

Alternative & Promising Directions

Fresh approaches that deliberately **avoid** the well-trodden path (learn pairwise edge-compatibility → greedy / simulated-annealing assembly). If that path has already stalled, pick from these — each attacks a *different* part of the problem.

Team brainstorm · reconstruct a shuffled & per-fragment-degraded 24×24 jigsaw · metric = mean SSIM

Why we need different ideas. The intuitive pipeline — a CNN that scores "does B sit next to A?", then greedily/SA-assemble — hits a wall: heavy *per-fragment* degradation (noise $\sigma 40-55$ + JPEG q35 + independent brightness/contrast) **destroys the fine edge continuity** those methods depend on. The most confident pairwise matches end up ~50% correct, and a 576-piece assembly needs ~90%. So the ideas below either (A) build a **more robust representation**, (B) use a **fundamentally different assembly algorithm**, or (C) learn the whole thing in a **different paradigm**.

Cheapest first (do these before the big bets): B2 (spectral layout) reuses any compatibility matrix you already have and could replace the whole solver in an afternoon; **A1 (foundation-model features)** is inference-only and tests whether representation robustness is the real bottleneck. Evaluate everything with the same yardsticks: *best-buddy precision*, *neighbour accuracy*, and final *SSIM* — so results are comparable across ideas.

A · Stronger / different fragment representations

A1 · Foundation-model features (DINOv2 / CLIP / SAM encoder)

low effort

Pitch. Don't train a compatibility net from scratch — embed each fragment (and its edge strips) with a large *pretrained* vision encoder that is already robust to noise/JPEG/contrast, and match on those features.

Why different / why it may work here. Our from-scratch CNN plateaus because 20×20 corrupted tiles carry little learnable signal; foundation features are trained on billions of images and are far more invariant to exactly this degradation. If they beat 0.48 best-buddy precision out of the box, that reframes the whole project.

First step. Upsample fragments (or take patch tokens), extract edge-strip descriptors, compute best-buddy precision — the same gauge as the main pipeline.

A2 · Coarse-to-fine on degradation-robust low frequencies

low effort

Pitch. Noise/JPEG kill high frequencies, but **low-frequency structure survives** (mean colour, coarse gradient). Solve a robust *coarse* layout on low-freq descriptors first, then refine placement locally.

Why different / why it may work. It never relies on the fine seam continuity that the degradation destroys; the coarse solve strongly constrains the fine solve, and coarse descriptors are far less ambiguous.

First step. 5×5-downsample each fragment, build coarse adjacency (even just average-colour continuity), solve a coarse arrangement, measure neighbour accuracy.

A3 · Joint photometric canonicalisation (un-shuffle the lighting, then match)

medium

Pitch. Each fragment has its *own* brightness/contrast/JPEG. Estimate a per-fragment photometric correction and the placement **jointly** (alternating), so that seam continuity *emerges* once neighbours are on a common scale.

Why different / why it may work. Naive per-fragment normalisation failed, but that ignored neighbours; a **joint** estimate (each fragment's gain/bias fitted to make its tentative seams continuous) is a strictly stronger formulation and directly targets the biggest matching-killer.

First step. Given a tentative assembly, least-squares fit each fragment's (gain, bias) to minimise seam discontinuity; iterate placement ↔ photometry.

B · Fundamentally different assembly algorithms

B1 · End-to-end differentiable assembly (Sinkhorn / Gumbel + Transformer)

bigger bet

Pitch. One network ingests all 576 fragments as a **set** and outputs a soft permutation (doubly-stochastic via Sinkhorn) mapping fragments → grid cells; trained end-to-end against the known arrangement of train images.

Why different / why it may work. Learns **global** assembly directly (cf. DeepPermNet / Visual Permutation Learning / Gumbel-Sinkhorn nets) — no hand-crafted pairwise scores, no greedy search. A global objective can exploit long-range and multi-fragment context that pairwise greedy structurally cannot.

First step. Transformer over fragment embeddings + 576 learned position queries + a Sinkhorn head; supervise with permutation cross-entropy (we have ground-truth perms for all train images).

B2 · Spectral / manifold layout (compatibility graph → 2-D coordinates)

low-medium

Pitch. Treat the fragment-fragment compatibility as a graph; recover each fragment's 2-D grid coordinate from the graph's **spectral embedding** — the leading Laplacian eigenvectors act as the (row, column) coordinates. Snap to the 24×24 grid.

Why different / why it may work. Non-combinatorial: assembly becomes a continuous eigenproblem that integrates **global** graph structure and tolerates many individually-wrong edges — exactly where greedy growth breaks. And it **reuses any compatibility matrix you already have**, so it's a cheap drop-in replacement for the SA solver.

First step. Symmetrise the existing scores into an affinity, take the smallest non-trivial normalised-Laplacian eigenvectors, check whether they linearly recover (row, col).

B3 · Row-then-column decomposition (2-D → two 1-D problems)

medium

Pitch. Cluster fragments into their 24 **rows** (row-mates share horizontal statistics), order each row as a 1-D chain, then order the 24 rows vertically. Two easy 1-D orderings instead of one hard 2-D assignment.

Why different / why it may work. Divide-and-conquer that exploits the grid structure; 1-D ordering is vastly easier and more robust than 2-D assignment, and a mistake stays local instead of cascading across the whole board.

First step. Use horizontal-only compatibility to grow best-buddy chains → partial rows; a second vertical pass stitches the rows.

C · Different learning paradigms

C1 · Global "assembly critic" energy + optimisation

medium-high

Pitch. Train a CNN that looks at a **whole** candidate assembled image and scores how coherent/real it is (a discriminator); then search placements to maximise this global score — or use it to re-rank a combinatorial solver's candidates.

Why different / why it may work. The critic sees the **global** picture, not isolated seams, so it can disambiguate flat/ambiguous tiles using object-level and long-range context that pairwise scoring misses.

First step. Train a real-vs-wrongly-assembled classifier (both are trivially generated from train perms); use its score/gradient to guide or re-rank placement.

C2 · Agentic / RL placement with backtracking

bigger bet

Pitch. An agent places fragments sequentially with a value function that permits undoing moves, trained by RL with reward = assembly correctness (dense, from known perms).

Why different / why it may work. A **learned** search policy that recovers from mistakes and learns a smart order (confident anchors first, ambiguous flats last) — unlike fixed greedy/SA heuristics.

First step. Imitation-learn from an optimal placement order on train puzzles first (cheap), then fine-tune with RL.

Evaluation harness for all of the above: report *best-buddy precision* and *neighbour accuracy* (frame-invariant, isolate assembly quality from the scorer), then final *SSIM* after an NLM denoise. A method is worth pursuing the moment its neighbour accuracy clears the current baseline — you do not need a full pipeline to know it's working. Ideas compiled for the PAZZLE team · Pasha883.